

Homework5: 带约束的0-1背包

Implementation

我们应用二维的dp数组：

$$\begin{aligned} dp[j][k] \\ j - cost \\ k - B/A \text{ 数量的差值} \end{aligned}$$

k 可能为负数，故引入 OFFSET=n，数组的实际下标为 k+OFFSET。

- 若商品为A

$$dp[j][k] = \max(dp[j][k], dp[j-p][k+1] + h)$$

- 若商品为B

$$dp[j][k] = \max(dp[j][k], dp[j-p][k-1] + h)$$

最后的答案即为：

$$\begin{aligned} k &\leq OFFSET \\ j &\leq cost \end{aligned}$$

满足以上条件的二维数组最大值。具体代码实现见 [DP.ipynb](#)

两种方式：

1. 每次复制一遍dp数组进行更新，会导致花销较大；
2. 倒序遍历，遵循0-1背包的代码风格，花销较小，但逻辑有点不清晰。

Performance test results and analysis

```
M = 10
最大快乐值：18
选择商品编号：[0, 1, 4]

M = 15
最大快乐值：26
选择商品编号：[0, 1, 3, 4]
```

时间复杂度：

$$O(n \cdot M \cdot n)$$

空间复杂度：

$$O(M \cdot n)$$

Difficulties encountered and solutions

很难想到有约束条件下背包问题的解决方式，即利用差值作为第二个维度。并且通过回溯来写出选取方案的时候，通过choice[pj][pk]比较难想，应该多做题多理解才能掌握dp的核心思路。